Loan Default Prediction — Corrected Notebook

Detailed Executive Summary of Data Preparation, Feature Engineering & Predictive Modeling

148,670	34	24.6%	78.8%	0.754
Loan Records	Original Features	Historical Default Rate	Final Model Accuracy	Final Model ROC-AUC

What makes this the "corrected" version: earlier iterations of this analysis engineered four simple yes/no "risk flags" (high DTI, high income, large loan, high LTV) and judged each purely by comparing default rates across the split. This notebook replaces that approach with three new candidate features — **payment_ratio**, **leverage_score**, and **credit_type_mismatch** — each engineered as a testable hypothesis, then run through the same statistical-significance and Information Value (IV) screening used for every other variable, rather than being assumed useful. Nothing is kept on intuition alone.

1. Overview & Objective

This notebook analyzes the same portfolio of 148,670 residential mortgage loans (34 original attributes) to identify what drives borrower default and to build a defensible predictive model. The target, Status, marks each loan Good (0) or Default (1): 112,031 loans (75.4%) are Good and 36,639 (24.6%) are Defaults. The pipeline runs through five stages in order — (1) clean and impute the raw data, (2) explore default-rate patterns across key business dimensions, (3) engineer and statistically test candidate features, (4) encode everything with Weight-of-Evidence (WOE) and screen it by Information Value (IV) and multicollinearity (VIF), and (5) fit and validate a logistic regression model — with an explicit, deliberate step to detect and remove data leakage before calling anything final.

2. Data Preparation & Cleaning

Three low-value fields were dropped outright: **ID** (a row identifier), **loan_limit**, **approv_in_adv**, and later **submission_of_application** — near-constant or non-predictive columns that add noise without signal.

Every other field with missing values was imputed using a **bucketed, distribution-aware median** approach rather than a single global mean/median, so that each filled-in value reflects borrowers in a similar financial position, not the portfolio average:

Field	Missing	Imputation logic
income	9,150	Median income within loan-amount decile buckets
rate_of_interest	36,439	Median rate within loan-amount × income quintile buckets
property_value	15,098	Median value within loan-amount decile buckets
Upfront_charges	39,642	Median charge within loan-amount × property-value quintile buckets
Interest_rate_spread	36,639	Median spread within rate-of-interest decile buckets
dtir1 (DTI)	24,121	Median DTI within loan-amount × income quintile buckets
LTV	15,098	Recomputed directly: (loan_amount / property_value) × 100, not imputed
term	41	Simple median fill
Neg_ammortization	121	Simple mode fill
loan_purpose	134	Simple mode fill

Two things are worth calling out. First, zeros in income, rate_of_interest, and Upfront_charges were treated as disguised nulls (a $0 income or 0% rate is not realistic) and reset to missing before imputation. Second, LTV is the one field that wasn't statistically imputed at all — since it's mathematically defined as loan_amount over property_value, every missing LTV was simply recalculated from those two fields once both were complete, which is more accurate than any bucketed estimate. After cleaning, the dataset had zero remaining nulls apart from a residual 200 rows in 'age' left untouched.

The cleaned data was split 70/30 into train and test sets (stratified on Status, random_state=42) **before** any WOE encoding, feature screening, or model fitting — keeping the test set fully unseen during every selection decision.

3. Exploratory Data Analysis — Default Rate by Segment

Default rates were profiled across region, loan purpose, credit-reporting agency, income, LTV and credit score to surface intuitive, business-relevant risk patterns before any modeling began.

Region

Region	Default Rate
North-East	30.4%
Central	27.5%
South	26.6%
North	22.5%

Loan purpose, income, LTV & credit score

Dimension	Highest-risk segment	Rate	Lowest-risk segment	Rate
Loan purpose	p2	33.1%	p4	23.0%
Income quartile	Lowest quartile	33.0%	Highest quartile	20.1%
LTV bucket	70–80%	33.4%	0–60%	15.6%
Credit score band	800–900	25.2%	700–750	24.1%

Credit score is essentially flat across every band (~24–25% default regardless of score), which already hints that raw Credit_Score will carry little weight later — confirmed statistically in Section 5. Income and LTV, by contrast, show clean, monotonic-ish gradients that make intuitive business sense. One credit-reporting category, **“EQUI,”** showed a default rate of **99.99%** — essentially every EQUI-reported loan defaulted, a near-perfect split that is a red flag for a data-quality anomaly or a proxy/leakage field rather than a genuine risk signal, so credit_type was treated with caution throughout.

4. Feature Engineering — Candidate Variables, Tested Not Assumed

One straightforward, clearly legitimate feature was engineered first: **loan_income_ratio** (loan_amount / income), a direct affordability measure visualized against Status via boxplot and carried forward as a normal predictor (IV = 0.47, see Section 6).

Three further variables were engineered as explicit **hypotheses** — combinations of existing fields that might carry information the individual fields don't, but which are not assumed useful until tested:

Candidate variable	Construction	Rationale for testing it
payment_ratio	loan_amount / term	loan_amount and term individually have weak IV (0.07 and 0.05) on their own, but combined they may describe payment burden — affordability — better than either alone.

leverage_score	dtir1 × loan_income_ratio	Compounds two already-legitimate leverage measures (existing debt burden and loan size relative to income) into a single combined score.
credit_type_mismatch	1 if credit_type ≠ co-applicant_credit_type, else 0	Flags whether the applicant and co-applicant were scored by different credit-reporting agencies — a relationship between two categorical fields that doesn't exist as information in either column alone.

Each candidate was carried through the same chi-square / Mann-Whitney significance tests as every other variable (Section 5), then WOE-encoded and screened against a minimum-usefulness bar of **IV > 0.02** — the same cutoff used to discard weak fields elsewhere in the pipeline (this is the same bar that eliminated the earlier high_ltv_flag / high_dti_flag style flags in the prior version of this analysis). All three candidates cleared it and were kept in the model feature set:

Candidate	Information Value	Verdict
leverage_score	0.304	Kept — comfortably clears the 0.02 bar
payment_ratio	0.148	Kept — clears the bar
credit_type_mismatch	0.107	Kept — clears the bar

leverage_score in particular was also checked for multicollinearity in Section 6, since it's built directly from dtir1 and loan_income_ratio — the concern being that it might just restate information those two fields already give the model, the same failure mode that got a similar engineered field (property_cushion_pct) dropped in the earlier version of this analysis. It passed: VIF around 3.2, well under the concern threshold.

5. Statistical Significance Testing

Chi-square tests (categorical variables, run against Status on the training set) covered Gender, loan_type, loan_purpose, Region, occupancy_type, business_or_commercial, Neg_ammortization, and the new credit_type_mismatch flag. **All eight were statistically significant (p < 0.05)**, led by Neg_ammortization and credit_type_mismatch itself (both p ≈ 0).

Mann-Whitney U tests (continuous variables) covered loan_amount, income, Credit_Score, property_value, LTV, dtir1, rate_of_interest, Interest_rate_spread, Upfront_charges, loan_income_ratio, payment_ratio, and leverage_score. **Every variable was significant except one: Credit_Score (p = 0.324)** — confirming the flat pattern seen across credit-score bands in Section 3. Income and property_value produced the strongest associations with default of any continuous field.

6. Weight of Evidence, Information Value & Multicollinearity

All predictors — raw and engineered — were WOE-encoded and ranked by IV to quantify standalone predictive strength. Values above ~0.5 are conventionally “suspiciously strong” and warrant a leakage check:

Variable	IV	Interpretation
Interest_rate_spread	5.36	Suspiciously strong — leakage risk
credit_type	4.55	Suspiciously strong — leakage risk
rate_of_interest	4.19	Suspiciously strong — leakage risk
LTV	2.85	Strong, legitimate predictor
Upfront_charges	2.72	Suspiciously strong — leakage risk
property_value	0.83	Strong, legitimate predictor
loan_income_ratio	0.47	Moderate, legitimate predictor
leverage_score (candidate)	0.30	Moderate — kept

dtir1	0.30	Moderate, legitimate predictor
payment_ratio (candidate)	0.15	Weak-moderate — kept
income	0.14	Weak, legitimate predictor
credit_type_mismatch (candidate)	0.11	Weak — kept
Neg_ammortization	0.11	Weak, legitimate predictor
Credit_Score	0.02	Not predictive (consistent with Mann-Whitney result)
year	0.00	No variance — dropped

Eight variables with negligible IV (year, open_credit, high-cardinality near-constants like Security_Type, Secured_by, total_units, construction_type, Credit_Worthiness, and interest_only) were dropped outright.

A first VIF pass flagged severe multicollinearity between **loan_type** (VIF ≈ 37.9) and **business_or_commercial** (VIF ≈ 37.1) — the two fields were carrying almost the same information. business_or_commercial was dropped, along with lump_sum_payment (VIF came back undefined/NaN due to near-zero variance in the training slice). After removing those two, the worst remaining VIF fell to about 3.6 (loan_amount), comfortably below the typical concern threshold of 5–10 — multicollinearity was effectively resolved.

7. Predictive Modeling & Data Leakage Correction

Two logistic regression models were built and deliberately compared, so that an inflated headline number couldn't be reported by accident.

Model A — initial model (leakage present)

Trained on every feature that survived IV/VIF screening, including Interest_rate_spread, rate_of_interest, credit_type and Upfront_charges. Performance was implausibly strong: **97.35% accuracy, 99.41% ROC-AUC, 98.82% Gini, and a KS statistic of 0.958.** Near-perfect separation like this is a textbook signature of target leakage: these four fields are effectively set at or after the underwriting decision, so a model using them isn't really predicting default — it's reading the outcome off fields that already encode it.

Model B — final, leakage-corrected model

Interest_rate_spread, rate_of_interest, credit_type and Upfront_charges were removed and the model retrained on the remaining legitimate, pre-decision fields (including all three surviving candidate variables). This produced realistic, decision-usable performance:

Metric	Good class (0)	Default class (1)	Overall
Precision	80.4%	65.8%	—
Recall	95.0%	29.2%	—
F1 score	87.1%	40.5%	—
Accuracy	—	—	78.8%

ROC-AUC: **0.7535** | Gini coefficient: **0.5070**

At the default 0.5 threshold the model is conservative: it correctly flags 95% of good loans but catches only 29.2% of actual defaults. Using the **KS-optimal threshold of 0.304** instead (max KS = 0.388) trades some overall accuracy (78.8% → 75.0%) for materially better default detection — recall on defaults rises from 29.2% to **58.5%**, at the cost of default-class precision falling from 65.8% to 49.4%. Which threshold to use is a business call: 0.5 favors overall accuracy, 0.304 favors catching more true defaults.

8. Decile / Risk-Ranking Analysis

Test-set borrowers were scored by the final model and split into ten equal-sized risk deciles (Decile 1 = highest predicted risk). This is arguably the most operationally useful output of the model, since it works independent of whichever classification threshold a risk team ultimately picks:

Decile	Total	Defaults	Good	Default Rate	Cumulative Default %
1 (highest risk)	4,460	2,990	1,470	67.0%	27.2%
2	4,460	2,187	2,273	49.0%	47.1%
3	4,461	1,326	3,135	29.7%	59.2%
4	4,459	1,052	3,407	23.6%	68.7%
5	4,460	862	3,598	19.3%	76.6%
6	4,460	667	3,793	15.0%	82.6%
7	4,460	551	3,909	12.4%	87.7%
8	4,460	533	3,927	12.0%	92.5%
9	4,460	439	4,021	9.8%	96.5%
10 (lowest risk)	4,461	385	4,076	8.6%	100.0%

The riskiest decile alone has a **67.0% default rate** — nearly two in three loans in that group eventually default — and captures **27.2%** of every default in the portfolio. The top 3 deciles together capture **59.2%** of all defaults while covering only 30% of the loan book. Practically, this means a risk team could route the riskiest ~30% of applications to enhanced manual underwriting and intercept roughly six in ten eventual defaults — a much more actionable use of the model than any single accept/reject cutoff.

Source: Loan_Default_Prediction_corrected.ipynb (102-cell Python/Jupyter notebook, dataset: 148,670 records, 34 original features). This document summarizes the corrected technical notebook for an executive audience; figures are drawn directly from the notebook's own printed outputs.